

Data Cleaning Process Report

ShopSphere Sales Analytics Project

Raw-to-Clean Transformation of the Sample Superstore Dataset

Jaymin

July 5, 2026

Overview

This report documents every data quality check performed on the raw `Sample_Superstore.csv` export, the issues that were discovered, and the exact fix applied for each one. The dataset represents order-line-level sales transactions for ShopSphere (mirroring the public Sample Superstore structure), covering Office Supplies, Furniture, and Technology products across the United States.

The cleaning process followed a repeatable sequence: **structural checks first** (duplicates, nulls), **then** format and consistency checks (dates, text fields, identifiers), **then** a final re-validation pass once fixes were applied. Each step below states what was checked, what was found, and what was done about it.

Final Outcome

Starting from a 9,994-row raw export, six categories of issues were investigated. Two were confirmed to be **valid business/structural characteristics of the data** (not errors), and four were **genuine data quality issues that required correction**. The final file, `Sample_Superstore_final.csv`, is fully validated and ready for SQL import.

1 Step 1: Structural Integrity Checks

Before touching any values, the dataset was checked for the two most basic data quality problems: exact duplicate rows and missing values.

1.1 Exact Duplicate Row Check

The full dataset (all 19 columns) was checked for rows that are 100% identical to another row.

• Issue Found: Duplicate Rows?

None expected to be ruled out blindly – verified directly against the raw file.

```
df = pd.read_csv('Sample_Superstore.csv', encoding='latin1')
print('Full duplicate rows:', df.duplicated().sum())
```

Result: 0 exact duplicate rows found out of 9,994. No fix required.

1.2 Order ID Repetition – Investigated, Not an Error

A separate check showed that `Order ID` repeats across **4,985 of 9,994 rows** (only 5,009 unique `Order IDs` exist). At first glance this looks like a duplication problem.

```
print('Duplicate Order IDs (rows):', df['Order ID'].duplicated().sum())
print('Unique Order IDs:', df['Order ID'].nunique())
```

Root Cause

This is expected behaviour, not corrupted data. One customer order can contain *multiple products*, and each product line gets its own row (with a unique `Row ID`) while sharing the same `Order ID`, `Customer ID`, and order date. For example, one order was found to contain 14 separate product lines, all sharing a single `Order ID`.

Conclusion: The grain of this dataset is *order line item*, not *order*. `Order ID` is intentionally

non-unique. No fix applied – this was documented in the data dictionary instead.

1.3 Missing / Null Value Check

Every column was checked for null or missing values.

```
print(df.isnull().sum())
```

Result: Zero nulls across all 19 columns and 9,994 rows. No fix required.

1.4 Range Checks on Numeric Fields

Sales, Quantity, and Discount were checked for invalid negative values, and Discount was checked to ensure it never exceeds 1 (i.e. stored as a proper 0–100% fraction).

Result: 0 negative Sales, 0 negative Quantity, 0 negative Discount, and 0 rows with Discount > 1. All numeric ranges were valid as-is.

2 Step 2: Date Format Standardization

2.1 Issue Found: Mixed Date Formats

• Issue Found: Mixed Date Formats in Order Date / Ship Date

The Order Date and Ship Date columns contained **two different string formats mixed within the same column**:

- **Dash format, zero-padded** – e.g. 11-08-2016 (4,042 rows in Order Date)
 - **Slash format, not zero-padded** – e.g. 6/16/2016 (5,952 rows in Order Date)
- The same split existed in Ship Date (3,898 dash-format / 6,096 slash-format rows).

2.2 Verifying the Date Order (Day-first vs. Month-first)

Because both parts of the dash-format dates only ever ranged 1–12, it was not possible to tell by inspection alone whether the format was MM-DD-YYYY or DD-MM-YYYY. This was resolved logically: Ship Date must always occur *on or after* Order Date. Both interpretations were tested against that rule.

```
# Hypothesis A: MM-DD-YYYY
od_A = pd.to_datetime(both['Order Date'], format='%m-%d-%Y')
sd_A = pd.to_datetime(both['Ship Date'], format='%m-%d-%Y')
delta_A = (sd_A - od_A).dt.days

# Hypothesis B: DD-MM-YYYY
od_B = pd.to_datetime(both['Order Date'], format='%d-%m-%Y')
sd_B = pd.to_datetime(both['Ship Date'], format='%d-%m-%Y')
delta_B = (sd_B - od_B).dt.days
```

Interpretation	Shipping delta (days)	Plausible?
MM-DD-YYYY	0 – 7 days (avg. 3.5)	Yes – matches real Ship Modes
DD-MM-YYYY	0 – 214 days (avg. 107)	No – unrealistic for retail

Conclusion: Both the dash-format and slash-format dates represent **MM-DD-YYYY** – there was no day/month ambiguity, only a formatting inconsistency (separator character and zero-padding).

2.3 2.3 Fix Applied: Standardized Date Format

✓ Fix Applied: Standardize All Dates to a Single MM-DD-YYYY Format

Pandas' mixed-format date parser was used to correctly interpret *both* the dash and slash variants in a single pass, then all dates were re-written in one consistent, zero-padded format.

```
df['Order Date'] = pd.to_datetime(df['Order Date'], format='mixed',
    dayfirst=False)
df['Ship Date'] = pd.to_datetime(df['Ship Date'], format='mixed',
    dayfirst=False)

df['Order Date'] = df['Order Date'].dt.strftime('%m-%d-%Y')
df['Ship Date'] = df['Ship Date'].dt.strftime('%m-%d-%Y')
```

Verified result: 100% of rows in both Order Date and Ship Date now match the single pattern MM-DD-YYYY (9,994 / 9,994 rows).

3 Step 3: Negative Profit Investigation

• Issue Found: Negative Values Found in Profit

1,871 of 9,994 rows (18.7%) had a negative Profit value, ranging as low as **-\$6,599.98**.

```
neg = df[df['Profit'] < 0]
print('Negative profit rows:', len(neg))
print('Min profit:', df['Profit'].min())
```

Root Cause – Not a Data Error

Profit is a derived business metric (Sales minus cost), so a loss on a sale is legitimate, especially at high discount levels. Inspecting the negative-profit rows showed a clear pattern: nearly all of them carried **discounts of 30%–80%**, concentrated in Furniture (Tables, Bookcases) and Office Supplies (Appliances, Binders). For example, a \$3,083 bookcase sold at a 50% discount resulted in a \$1,665 loss.

Conclusion: Negative Profit is a **valid business signal driven by discounting**, not a data quality defect. No fix was applied to the values themselves; the finding was documented for the analysis phase instead (e.g. as a potential KPI: “discount threshold at which orders become unprofitable”).

4 Step 4: Column Name Standardization

• Issue Found: Inconsistent Column Naming

Original column headers used mixed case and embedded spaces / hyphens (e.g. "Order ID", "Ship Mode", "Sub-Category"), which is unsafe for direct SQL import and inconsistent for programmatic use.

✓ Fix Applied: Standardize Column Names to snake_case

All column names were converted to fully lowercase **snake_case**, with spaces and hyphens replaced by underscores.

```
df.columns = [c.replace('-', '_').replace(' ', '_').lower() for c in df.columns]
```

Result: e.g. "Order ID" → order_id, "Ship Mode" → ship_mode, "Sub-Category" → sub_category. Verified: no remaining spaces, hyphens, or SQL reserved-word conflicts among the 19 column names.

5 Step 5: Additional Data Quality Issues

A deeper re-validation pass on the partially-cleaned file surfaced four further issues that a first-pass check does not typically catch, since they are invisible in a normal spreadsheet view.

5.1 5.1 Non-Breaking Space Characters in Product Name

• Issue Found: Hidden Unicode Characters (\xa0)

278 rows (48 unique product names) in product_name contained a *non-breaking space* character (Unicode U+00A0) in place of a normal space – e.g. "Konftel 250 Conference_{phone}- Charcoal black" visually looks identical to a normal space but is a different byte sequence. This can silently break exact-match WHERE clauses and GROUP BY aggregations in SQL.

```
mask = df['product_name'].apply(lambda x: '\xa0' in str(x))
print('Rows with non-breaking space:', mask.sum())
```

✓ Fix Applied: Replace Non-Breaking Spaces with Regular Spaces

```
df['product_name'] = df['product_name'].str.replace('\xa0', ' ',
                                                    regex=False)
```

Verified result: 0 rows remaining with non-breaking space characters.

5.2 5.2 Leading / Trailing Whitespace in Product Name

• Issue Found: Stray Whitespace

16 rows in product_name had leading or trailing whitespace, e.g. 'Iceberg Mobile Mega Data/Printer Cart ' (trailing space). This can cause the same product to be treated as two different values in grouping or joins.

✓ Fix Applied: Strip Leading/Trailing Whitespace and Collapse Internal Double Spaces

```
df['product_name'] = df['product_name'].str.strip()
df['product_name'] = df['product_name'].str.replace(r'\s+', ' ',
                                                    regex=True)
```

Verified result: 0 rows remaining with leading/trailing whitespace.

5.3 Inconsistent Casing in Customer ID

• Issue Found: Mixed-Case Identifiers

43 rows (3 unique customer IDs) did not follow the standard XX-NNNNN identifier pattern because the second letter was lowercase: D1-13600, Co-12640, Dp-13240 instead of DL-13600, CO-12640, DP-13240. This did not create duplicate customer identities, but is inconsistent formatting that could cause mismatches in case-sensitive SQL joins (e.g. PostgreSQL).

```
mask = ~df['customer_id'].str.match(r'^[A-Z]{2}-\d{5}$')
print('Rows with inconsistent ID casing:', mask.sum())
```

✓ Fix Applied: Standardize Customer ID to Uppercase

```
df['customer_id'] = df['customer_id'].str.upper()
```

Verified result: 0 rows remaining with non-standard casing.

5.4 (Optional) Mixed-Case Column Names Fully Normalized

• Issue Found: Residual Mixed Casing After Initial Rename

The first column-renaming pass (Step 4) lowered only the first letter of each header, leaving inconsistent internal casing such as `order_ID`, `ship_Date`, and `sub_Category` alongside fully lowercase names like `segment` and `sales`.

✓ Fix Applied: Fully Normalize All Column Names to Lowercase snake_case (Optional Polish)

```
df.columns = [c.replace('-', '_').replace(' ', '_').lower() for c in df.columns]
```

Verified result: All 19 column names are now uniformly lowercase `snake_case` (e.g. `order_id`, `ship_date`, `sub_category`, `product_name`), matching standard SQL naming convention.

5.5 Note: Remaining Special Characters (No Fix Needed)

After the fixes above, 53 rows in `product_name` still contained non-ASCII characters. These were inspected individually and confirmed to be **legitimate typographic characters in real product names** – curly quotes (e.g. "Pillow Soft"), inch/fraction symbols (e.g. 5 3/6"), and accented letters (e.g. *Résumé* Paper). These are correct as-is; most SQL engines store text as UTF-8/Unicode natively, so no further action was taken.

6 Final Validation Summary

The table below summarizes every check performed, from the initial structural pass through the final polish, comparing the state before and after cleaning.

Check	Before Cleaning	After Cleaning
Exact duplicate rows	0	0 (unchanged – never an issue)
Order ID repetition	4,985 rows share an ID	Documented as expected (multi-line orders)
Null values	0	0 (unchanged – never an issue)
Negative Sales/Qty/Dis-count	0	0 (unchanged – never an issue)
Date formats	2 mixed formats (dash + slash)	1 consistent MM-DD-YYYY format
Negative Profit	1,871 rows (18.7%)	Retained – valid business signal
Column naming	Spaces, mixed case	Fully lowercase snake_case
Non-breaking spaces	278 rows affected	0 remaining
Leading/trailing white-space	16 rows affected	0 remaining
Customer ID casing	43 rows / 3 IDs affected	0 remaining

7 Conclusion

The raw `Sample_Superstore.csv` export was taken through a structured, repeatable cleaning pipeline: structural checks (duplicates, nulls) → format checks (dates) → business-logic checks (negative profit) → naming standardization → a deep re-validation pass that surfaced hidden Unicode and casing issues invisible to a normal spreadsheet view.

Final Status

The resulting file, `Sample_Superstore_final.csv`, has zero nulls, zero duplicate rows, a single consistent date format, standardized **snake_case** column names, and clean, consistently-formatted text and identifier fields. It is validated and ready for import into a SQL database for the ShopSphere sales and profitability analysis.